

SECURITY INCIDENT ANALYSIS REPORT

AI Supply Chain Attack

Hugging Face Malicious Repository Campaign

How Attackers Weaponized a Trusted AI Platform to Steal Your Data

Severity	CRITICAL
Attack Type	AI Supply Chain + Infostealer
Platform	Hugging Face (AI Model Hub)
Target	AI Developers, ML Engineers, Researchers, Students
OS Affected	Windows
Estimated Downloads	~244,000 in under 18 hours

Author Krish Gupta Cybersecurity Researcher Penetration Tester Offensive Security Enthusiast	Report Details Date: May 2026 Classification: Educational / Public
--	---

This report is created for educational and awareness purposes only. All information is sourced from publicly available security research.

SECTION 1: THE BIG PICTURE — WHAT HAPPENED?

The Story in Two Lines

Attackers created a fake AI tool on Hugging Face, made it look exactly like a real OpenAI product, got it trending with 244,000+ downloads — and used it to silently steal passwords, tokens, and crypto wallets from every developer and researcher who ran it. No virus alert. No warning. Nothing.

1.1 Why Is This Different From a Normal Virus?

Most people think malware looks like this:

"Virus detected! Your PC is infected!"

This attack looked like this:

"Welcome to the OpenAI Privacy Filter. Run loader.py to get started."

That is the fundamental shift. **The user was the one who ran the malware — willingly — because they had absolutely no reason to suspect anything was wrong.** This is what makes AI supply chain attacks so uniquely dangerous.

1.2 Who Was Targeted?

This was not a random attack on everyday users. It specifically targeted:

- AI/ML developers using Hugging Face to download models and tools
- Researchers exploring new AI releases
- Students learning AI who trust trending repositories
- Security professionals testing AI tools
- Anyone searching for 'OpenAI Privacy Filter' on the platform

Real-World Analogy — The Fake Medicine Store

Imagine a shop opens next to a famous pharmacy.
Same logo. Same colours. Same product names. Same packaging.
But inside the medicine boxes — poison.

That is exactly what happened on Hugging Face.
Same name. Same description. Same model card as the real OpenAI repo.
But inside the code — malware.

SECTION 2: UNDERSTANDING THE PLATFORM — WHAT IS HUGGING FACE?

Before understanding the attack, you need to understand why Hugging Face was such a perfect target.

2.1 What is Hugging Face?

Hugging Face is the GitHub of the AI world. It is a platform where:

- AI researchers publish trained models (like GPT, image generators, etc.)
- Developers share datasets for training AI
- Teams release tools, demos, and code
- Anyone can download and use what others have built

It has millions of users and is trusted by Google, Meta, Microsoft, and independent researchers worldwide.

2.2 Why Did People Trust the Fake Repo?

Trust Signal	Why It Worked
Looked like an official OpenAI repository	OpenAI is one of the most recognized names in AI
244,000+ downloads	'That many people downloaded it — must be safe'
667 likes and top trending position	Trending = popular = legitimate (false assumption)
Same description as the real repo	Copied word for word — impossible to tell apart quickly
Ran from a platform people already trust	Nobody suspects malware on Hugging Face

The Psychology of Trust

Human brains use shortcuts. One powerful shortcut: 'if many people trust it, it must be safe.' Attackers know this. They artificially inflated download counts and likes to exploit this bias. This is called Social Proof Manipulation — and it works extremely well.

2.3 What is Typosquatting? (The Name Trick)

The fake repo name was: Open-OSS/privacy-filter

The real repo name was: openai/privacy-filter

Spot the difference? Most people scan quickly and miss it completely. This technique is called Typosquatting:

Real	Fake	What You Miss
openai/privacy-filter	Open-OSS/privacy-filter	'Open-OSS' instead of 'openai'
google.com	go0gle.com	Number zero instead of letter O
paypal.com	paypal.com	Capital I instead of lowercase l

One character. That is all it takes to trick thousands of people.

SECTION 3: THE COMPLETE ATTACK CHAIN — STEP BY STEP

This is the most important section. Every step will be explained simply — with what actually happened and why it was dangerous.

3.1 The Full Flow at a Glance

1	Victim finds the trending fake repo on Hugging Face
2	Victim runs loader.py — looks harmless, is not
3	Python silently contacts a remote server
4	JSON payload fetched with hidden PowerShell commands
5	PowerShell runs invisibly in the background
6	start.bat downloaded — begins privilege escalation
7	Windows Defender exclusions added — antivirus blinded
8	Final infostealer (Rust-based) installed and running
9	Passwords, cookies, tokens, crypto wallets stolen
10	All stolen data sent silently to attacker's server

3.2 Step 1 — The Bait: A Fake Trending Repository

Victim opens Hugging Face and sees the 'OpenAI Privacy Filter' trending at the top. It has:

- 244,000+ downloads (artificially inflated)
- 667 likes
- The exact same description as the real OpenAI repository
- A professional-looking README with instructions

The README says something like:

```
# OpenAI Privacy Filter
# Get started:
python loader.py
```

Victim thinks: 'Great, new OpenAI tool. Let me try it.' And runs the command.

Why This Works

AI tools commonly require running Python scripts to set up.

'python loader.py' is completely normal and expected in this context. There is nothing visually suspicious. The code even has fake AI logic and comments inside to make it look legitimate if anyone glances at it.

3.3 Step 2 — loader.py: The Trojan Horse

What is loader.py?

It is a Python file. On the surface, it has fake ML/AI code with proper comments and structure. Anyone quickly looking at it would think: 'Just a setup script.'

But hidden inside, it does five dangerous things:

1. Disables SSL certificate verification
2. Contacts a remote attacker-controlled server
3. Fetches a JSON payload with hidden commands
4. Decodes Base64-encoded instructions
5. Executes PowerShell silently

What is SSL and why disabling it matters?

Normally, when your browser or Python connects to a website, it verifies the website's identity certificate — like checking an ID card. This prevents fake servers from impersonating real ones.

Malware disables this check with one line:

```
requests.get(url, verify=False) # SSL check disabled
```

Now Python will connect to ANY server — even a fake malicious one — without questioning it. The attacker's server can masquerade as anything.

What is Base64 Encoding and why use it?

Base64 is a way to convert text into a different format. It is not encryption — anyone can decode it — but it makes commands look like random gibberish at a glance.

Format	Example
Normal (visible)	https://attacker-server.com/payload.exe
Base64 (hidden)	aHR0cHM6Ly9hdHRhY2tldi1zZXJ2ZXluY29t

Why use it?

- Humans scanning code will not notice it as a suspicious URL
- Simple antivirus scanners that look for obvious keywords will miss it
- It adds one more layer of confusion between the analyst and the attack

3.4 Step 3 — JSON Payload: Commands From a Remote Master

What is a JSON payload?

JSON is a standard data format used by apps, APIs, and settings files. Completely normal in software development.

Example of normal JSON:

```
{ "name": "krish", "version": "1.0" }
```

But in this attack, JSON was used to carry hidden commands:

```
{ "cmd": "powershell -WindowStyle Hidden -EncodedCommand ..." }
```

Or a URL to download the next stage:

```
{ "url": "https://malicious-domain.com/stage2.exe" }
```

Why Remote JSON Delivery is So Smart

The malware in the repo did NOT contain the actual dangerous payload.
The real instructions were stored on an external server.

This means:

- If someone inspects the GitHub/Hugging Face code: it looks relatively clean
- Attackers can change the payload any time without updating the repo
- The payload could be turned on/off like a switch
- Detection by static scanning is much harder

3.5 Step 4 — PowerShell: The Invisible Executor

What is PowerShell?

PowerShell is Windows' built-in automation and scripting engine. Administrators use it for:

- Installing software
- Changing system settings
- Running scheduled tasks
- Managing files and networks

Attackers use it because:

- It is already installed on every Windows machine
- Windows trusts it completely — it is an official tool
- It can do almost anything on the system if given permissions
- It can run completely invisibly, with no window shown to the user

How was it run invisibly?

```
powershell.exe -WindowStyle Hidden -EncodedCommand BASE64_HERE
```

-WindowStyle Hidden = No popup. No CMD window. Nothing visible to the user.

-EncodedCommand = The actual command is Base64 encoded so it is harder to read in process logs.

Real-World Analogy

Imagine someone enters your house, does everything they need to do, and leaves — while you are sitting in the living room, completely unaware. They used the back door. They moved quietly. No alarms triggered.

PowerShell running in hidden mode is that back door.

3.6 Step 5 — Privilege Escalation: Becoming the Admin

What is Privilege Escalation?

On Windows, there are different levels of power:

User Level	Admin Level
Can read and write your own files	Can change system settings
Cannot modify system programs	Can install/uninstall anything
Cannot change antivirus settings	Can disable or configure antivirus
Cannot add startup programs system-wide	Can add persistence anywhere

Privilege escalation means: going from the left column to the right column — without authorization.

How did the malware attempt this?

Via the downloaded start.bat batch file. Methods that may have been used include:

- User was already an administrator on their machine (very common on personal/dev PCs)
- UAC (User Account Control) prompt appeared and user clicked 'Yes' thinking it was the AI tool
- Token abuse — stealing admin-level tokens from memory
- Scheduled task exploitation

What is the UAC Prompt?

```
Do you want to allow this app to make changes to your device?  
[ Yes ]    [ No ]
```

That familiar Windows popup. Most users click 'Yes' habitually, especially if they believe they are setting up a legitimate tool. That single click handed the malware admin privileges.

3.7 Step 6 — Windows Defender Exclusions: Blinding the Guard

What are Defender Exclusions?

Windows Defender (the built-in antivirus) has a legitimate feature called Exclusions. It tells the antivirus:

"Do not scan this folder / this file / this program."

Legitimate uses of this feature:

- Developers exclude their build folders (antivirus can slow compilation)
- IT admins exclude trusted corporate software from scanning
- Gamers sometimes exclude game folders for performance

What the malware did:

```
Add-MpPreference -ExclusionPath "C:\Users\User\AppData\Roaming\Updater\"
```

Translation: 'Windows Defender, please ignore this folder completely.'

The malware installed itself in that excluded folder. From that moment, Defender would not scan, detect, or alert on anything in that location.

This is LOLBins — Living Off The Land

LOLBins = Legitimate tools used maliciously.

The malware did NOT hack Windows Defender.

It used Windows Defender's own official settings to disable itself.

Using legitimate tools for malicious purposes is much harder to detect because the system sees official commands, not obvious 'hacker tools.'

Other common LOLBins: powershell.exe, cmd.exe, mshta.exe, rundll32.exe

3.8 Step 7 — The Infostealer: What Was Actually Stolen

What is an Infostealer?

Infostealer = Information + Stealer. A specific type of malware built for one purpose: silently steal data and send it to the attacker.

This one was built in Rust — a modern programming language — making it:

- Faster and more efficient than older malware
- Harder to analyse (fewer automated tools for Rust malware)
- Increasingly popular among professional threat actors

Exactly What Was Stolen:

Data Type	How It Was Stored	Why Attackers Want It
Browser passwords	SQLite databases (Login Data file)	Direct account access

Data Type	How It Was Stored	Why Attackers Want It
Session cookies	Browser cookie databases	Login without needing password
Autofill data	Browser profile files	Identity info, addresses, card data
Discord tokens	Local app storage	Take over Discord account instantly
Crypto wallet files	Wallet data + seed phrases	Drain cryptocurrency
Browser sessions	Active session tokens	Access already-logged-in accounts
Sensitive documents	File system scan	Credentials, keys, private info

Why Are Session Cookies the Most Dangerous?

Most people think passwords are the main target. But session cookies are worse.

Scenario	What Happens
You log into Gmail	Gmail gives your browser a session cookie: SESSION=xyz123
Browser stores that cookie	Every request uses it to prove you are logged in
Malware copies that cookie	Attacker now has your active session token
Attacker uses your cookie	They are logged into your Gmail — instantly
No password needed	And sometimes even MFA/2FA cannot stop this

How Were Chrome Passwords Stolen Despite Encryption?

Chrome encrypts saved passwords using Windows DPAPI (Data Protection API).

Here is the key: DPAPI decrypts data automatically for the currently logged-in user.

Since the malware was running AS that same logged-in user — Windows happily decrypted the passwords for it. No crack needed. Windows did the decryption itself.

How Did Stolen Data Reach the Attacker?

Via one of these standard channels:

Method	How It Works
C2 Server (Command & Control)	Attacker's own server — malware calls home and uploads data
Telegram Bot	Data sent directly to attacker's Telegram inbox
Discord Webhook	Data posted to attacker's Discord server channel
HTTP POST request	Simple web request to upload a file to attacker's URL

SECTION 4: WHY IS THIS ATTACK SO DANGEROUS?

4.1 The Six Reasons This Was Uniquely Threatening

1. Trusted Platform Abuse

Hugging Face is genuinely trusted by millions. Attackers did not need to trick people into visiting a suspicious website — the victims were already on a platform they trusted and use daily.

2. AI Hype as a Weapon

2025–2026 is peak AI hype. Developers and researchers are constantly looking for new AI tools, models, and releases. The urgency to try new things overrides caution.

The Hype Trap

Attacker's logic: 'They are so excited about AI that they will run anything claiming to be OpenAI.'
The hype itself was the vulnerability.

3. 'Open Source = Safe' is a Dangerous Assumption

Many developers believe: 'The code is public, so anyone would have spotted malware in it by now.'

This attack exploited that assumption. The dangerous parts were fetched remotely at runtime — they were never in the repo for anyone to inspect statically.

4. Multi-Stage Attack Chain

It was not a single malicious file. It was a chain:

```
Python script → Remote JSON → PowerShell → BAT file → Rust EXE
```

Each stage looks benign without the previous context. Security tools analysing individual files may not see the full picture.

5. No Exploit Required — The User Did Everything

There was no CVE. No zero-day. No vulnerability in Windows or Hugging Face was exploited.

The attacker's only tool was: trust + branding + curiosity.

The victim ran the malware themselves.

6. Scale

244,000 downloads in under 18 hours. Even if only 1% of downloaders ran it — that is 2,400 compromised machines. Likely many more.

4.2 This is the Future of Malware Distribution

As npm, pip, and GitHub became malware distribution vectors over the past decade — Hugging Face is now joining that list. Expect to see this more:

- Fake AI models with embedded loaders
- Poisoned training datasets
- Malicious LoRA adapters and fine-tuned model weights
- Trojanized inference scripts
- Prompt injection via public model cards

The Ecosystem Is Now a Target

npm had malicious packages. pip had malicious packages.

Now Hugging Face has malicious models.

The AI supply chain is the new frontier for attackers.

SECTION 5: FOR THE NON-TECHNICAL READER — REAL-WORLD ANALOGIES

If any part of this still feels abstract, these analogies map every concept to real life.

Technical Concept	Real-World Analogy
Hugging Face	The App Store — trusted, popular, where everyone goes for tools
Fake repo with same name	A counterfeit App Store app with the same icon and name as WhatsApp
loader.py running	You tap 'Open' on the fake app, trusting the store
Remote JSON payload	The app phones home and downloads instructions from the attacker
PowerShell hidden execution	An invisible person in your house following those instructions
UAC prompt + user clicks Yes	You hand the invisible person your house keys
Defender exclusions	You tell your home security camera: 'ignore that corner of the room'
Session cookies stolen	Someone copies your hotel room keycard — they can now enter any time
C2 server	The attacker's remote headquarters receiving all stolen items
Infostealer	A pickpocket who empties your wallet, copies your ID, and photographs your bank cards

SECTION 6: COULD IT BE DETECTED? (BLUE TEAM VIEW)

6.1 What Made Detection Hard

- No single file contained the full malicious payload — it was assembled at runtime
- PowerShell and batch files are normal — hard to distinguish from legitimate admin use
- Defender exclusions were added using official Windows APIs — not a bypass, a feature
- The initial Python script looked like a legitimate AI loader
- Remote payload delivery meant the repo itself could pass static analysis

6.2 Red Flags to Watch For

These Should Have Raised Alarms

verify=False ← SSL disabled in Python code
Encoded strings (Base64) used as URLs or commands
requests.get() fetching from unknown external domains
subprocess.run() or os.system() calling powershell.exe
-WindowStyle Hidden in any PowerShell command
Add-MpPreference -ExclusionPath ← Defender exclusion being added
A script adding itself to Windows startup / scheduled tasks
Outbound connections to non-standard domains after running

6.3 Detection Methods

Method	Approach
Static code analysis	Scan loader.py for Base64 strings, verify=False, subprocess calls
Network monitoring	Alert on unexpected outbound connections after running AI tools
PowerShell logging	Enable Script Block Logging — logs all PowerShell commands
Process monitoring	Flag powershell.exe launched with -WindowStyle Hidden
Defender event logs	Alert on Add-MpPreference commands from unexpected processes
Sandbox detonation	Run suspicious repos in isolated VM — observe runtime behaviour
Behavioural EDR	Modern EDR tools catch multi-stage execution chains

SECTION 7: IF YOU DOWNLOADED OR RAN THE REPOSITORY

URGENT — Take These Steps Immediately

Security researchers strongly recommend treating the machine as fully compromised. Do not attempt to 'clean' it — reinstall the operating system.

7.1 Immediate Actions

1	Disconnect the machine from the internet immediately
2	Do NOT log into any accounts from this machine
3	Rotate ALL passwords from a clean, unaffected device
4	Revoke and regenerate all API keys and access tokens
5	If you hold cryptocurrency — move funds to a new wallet with a new seed phrase now
6	Invalidate all active browser sessions (sign out of everything from another device)
7	Enable MFA/2FA on every account if not already enabled
8	Wipe and reinstall the operating system — do not just run antivirus
9	Report the incident to your organization's security team if applicable

SECTION 8: HOW TO PROTECT YOURSELF GOING FORWARD

8.1 Before Running Any Repository

6. Check the repository owner name character by character — typosquatting is subtle
7. Look at the account creation date — malicious accounts are often brand new
8. Search for the tool independently — does it appear on the real organization's official website?
9. Read the code before running it — even a 5-minute glance can catch obvious malware
10. Check if the downloads/likes look artificially inflated — 244k downloads overnight is suspicious

8.2 Technical Safeguards

- Always run unknown code in a virtual machine or sandbox first
- Enable PowerShell Script Block Logging on Windows
- Use an EDR (Endpoint Detection & Response) tool, not just antivirus
- Enable Windows Defender Tamper Protection — prevents malware from disabling it
- Monitor outbound network connections from development machines
- Do not run random repositories with admin / sudo privileges

8.3 For AI/ML Developers Specifically

- Prefer official organization accounts on Hugging Face — verify the URL path matches
- Use model cards and community tabs to check for user reports of suspicious behaviour
- Consider using Hugging Face's model scanning features where available
- Treat AI model loaders like any other third-party code — review before running
- Report suspicious repositories to Hugging Face immediately

The Golden Rule

'The code is open source so it must be safe' is one of the most dangerous assumptions in security. Open source means anyone CAN review it. It does not mean anyone DID. Always verify before you trust.

SECTION 9: GLOSSARY — EVERY TERM EXPLAINED

Term	Plain-English Meaning
Hugging Face	The GitHub of AI — a platform to share and download AI models and tools
Supply Chain Attack	Attacking the tools and platforms developers trust, not the developers directly
Typosquatting	Creating a fake account/website with a name nearly identical to a trusted one
loader.py	The Python script that pretended to be an AI setup tool but was actually malware
SSL Verification	Checking that a website's identity certificate is genuine — like verifying an ID card
Base64 Encoding	A way to convert text into a scrambled-looking format — not encryption, just obfuscation
JSON Payload	Data format used here to carry hidden commands from the attacker's server
C2 Server	Command & Control — the attacker's remote server that sends commands and receives stolen data
PowerShell	Windows' built-in automation tool — powerful, trusted, and abused by attackers
-WindowStyle Hidden	PowerShell flag that runs it with no visible window — completely invisible to the user
Privilege Escalation	Going from normal user permissions to administrator level — without authorization
UAC Prompt	Windows' 'Do you want to allow changes?' popup — clicking Yes grants admin rights
LOLBins	Living Off The Land Binaries — using legitimate Windows tools for malicious purposes
Defender Exclusions	Telling Windows antivirus to ignore specific folders or files
Infostealer	Malware designed specifically to steal passwords, cookies, and credentials
Session Cookie	A token your browser holds after login — stealing it allows login without a password
DPAPI	Windows' built-in encryption for user data — automatically decrypts for the current user

Term	Plain-English Meaning
Rust	A modern programming language — increasingly used by attackers for efficient, hard-to-analyse malware
Sandbox	An isolated virtual environment to run suspicious code safely without risk to the real system
EDR	Endpoint Detection & Response — advanced security tool that monitors behaviour, not just known signatures
Social Engineering	Manipulating people psychologically rather than exploiting technical vulnerabilities
Social Proof Manipulation	Faking popularity (downloads, likes) to make something appear trustworthy

SECTION 10: QUICK REFERENCE

10.1 Attack Summary Table

Attack Phase	What Happened	Real-World Equivalent
Lure	Fake trending OpenAI repo on Hugging Face	Counterfeit medicine with real brand packaging
Execution	Victim runs loader.py	Swallowing the medicine without reading the ingredients
Communication	Remote JSON payload fetched	Pill dissolves and phones home for further instructions
Escalation	start.bat + UAC + admin rights	Handing your house keys to the person inside
Blinding	Defender exclusions added	Telling your security camera to ignore one room
Payload	Rust info stealer installed	The actual robbery begins
Exfiltration	Data sent to C2 server	Stolen goods carried out the back door

10.2 Red Flag Checklist

▶	verify=False in Python code — SSL disabled
▶	Base64 encoded strings used as commands or URLs
▶	requests.get() calling external unknown domains
▶	subprocess.run() or os.system() launching powershell.exe
▶	-WindowStyle Hidden flag in any script
▶	Add-MpPreference -ExclusionPath in any command
▶	Repository with suspiciously inflated download numbers
▶	Account name very similar to a trusted organization but not exact
▶	Account created very recently with many published models
▶	Installer script that requests admin/UAC privileges

10.3 The One-Liner

This Attack in One Sentence:

Attackers created a fake OpenAI repository on Hugging Face, made it trend with artificially inflated downloads, then used a chain of Python → remote JSON → PowerShell → BAT → Rust infostealer to silently steal passwords, session cookies, Discord tokens, and crypto wallets from every developer who trusted the platform and ran the code.

[END OF REPORT — EDUCATIONAL PURPOSE ONLY]

Author: Krish Gupta | Cybersecurity Researcher | Penetration Tester | Offensive Security
This document is created for educational and awareness purposes only.